



Kernel Security: how2rootkit



Today

- Homework 4 is out!
- `callsyms`
- syscall hooking 101

/proc/kallsyms: The Kernel's Symbol Table

/proc/kallsyms is a virtual file that lists **every symbol in the running kernel** — functions, variables, and per-CPU data — with their runtime addresses.

```
$ cat /proc/kallsyms
....
ffff800080010000 T _text
ffff800080020000 T do_sys_openat2
ffff800080020180 T ksys_read
ffff800080020300 t do_vfs_ioctl
ffff80008002a000 D sys_call_table
```

Format: address type name [module]

Column	Meaning
address	Runtime virtual address (KASLR-adjusted)
type	Symbol type (uppercase = global, lowercase = local)
name	Symbol name as seen in source
[module]	Module name (absent for built-in symbols)

Key symbol types:

Type	Meaning
T / t	Text (code) — functions
D / d	Data — initialized global variables
B / b	BSS — uninitialized globals
R / r	Read-only data

Using kallsyms

- **Find hook targets:** `grep ' T do_sys_' /proc/kallsyms`
- **Verify addresses:** confirm `kprobe_lookup()` returned the right value
- **Detect table patches:** compare `sys_call_table` entries against expected addresses
- **Bypass GPL** (challenge mode): resolve any symbol from userspace, pass address to module

Who can read kalsyms?

- requires CAP_SYSLOG or root. Non-root sees addresses as 0000000000000000 (controlled by kptr_restrict).
- KASLR randomizes the base address each boot, so addresses change on every reboot.
- Our kernel does not have KASLR enabled for now.

kallsyms_lookup_name(): (Pre-5.7)

The kernel maintains a complete symbol table internally — every function name mapped to its runtime address. Before kernel 5.7, modules could query it directly:

```
#include <linux/kallsyms.h>

static int __init my_init(void)
{
    unsigned long addr;

    addr = kallsyms_lookup_name("do_sys_openat2");
    if (!addr) {
        pr_err("symbol not found\n");
        return -ENOENT;
    }

    pr_info("do_sys_openat2 is at %pK\n",
            (void *)addr);
    /* Now use addr for ftrace, etc. */
    return 0;
}
```

One function call. Name in, address out. KASLR-adjusted, always correct.

impl

- The kernel compiles with `CONFIG_KALLSYMS=y`
- This embeds a compressed symbol table into the kernel image itself.
- `kallsyms_lookup_name()` searches this table
 - it never touches `/proc/kallsyms` (that file is just a procfs interface to the same data).

Use case

- Feels like `dlsym` in the kernel (very loosely true)
- Resolve **any** symbol (exported or not)
- Find `sys_call_table`, internal functions, private data structures
- Bypass the `EXPORT_SYMBOL` / `EXPORT_SYMBOL_GPL` system entirely

The Unexport: Kernel 5.7

In February 2020, the kernel developers **unexported** `kallsyms_lookup_name()` from modules.

Why?

Google "nvidia linus torvalds"



Why?

- Non-GPL module developers would slap `MODULE_LICENSE("GPL")` on proprietary code,
- then call `kallsyms_lookup_name()` to access **any** kernel function — exported or not.
- This defeated the entire `EXPORT_SYMBOL` system that controls which kernel APIs are available to modules.

The function still exists inside the kernel. Modules just can't link against it anymore. ...but we are the kernel?

What does this mean for us?

```
/* This worked on kernel 5.6 */  
addr = kallsyms_lookup_name("do_sys_openat2");  
  
/* On kernel 5.7+: linker error – symbol not found */
```

5.7+

Direction	Function	Status
Name → address	<code>kallsyms_lookup_name()</code>	Unexported (modules can't use it)
Address → name	<code>sprint_symbol()</code>	Still exported (modules can use it)
All symbols	<code>/proc/kallsyms</code>	Still readable (root/CAP_SYSLOG)

- note that most kernel ship with kallsyms
- If it still exists, the lack of exported symbol doesn't really stop us

Workaround 1: The Kprobe

(credit: f0lg0 / TheXcellerator): `register_kprobe()` internally calls `kallsyms_lookup_name()` to resolve `.symbol_name`. The kprobe subsystem is privileged kernel code — it still has access to the unexported function.

So: register a throwaway kprobe, read the resolved address from `kp.addr`, unregister immediately. We never use the kprobe for tracing — just for symbol resolution.

Requirements:

- `CONFIG_KPROBES=y`
- `MODULE_LICENSE("GPL")` (kprobes API is GPL-only)

kprobe trick

trace_openat_ftrace.c, lines 60–70:

```
static unsigned long kprobe_lookup(const char *name)
{
    struct kprobe kp = { .symbol_name = name };
    unsigned long addr;

    if (register_kprobe(&kp) < 0)
        return 0;
    addr = (unsigned long)kp.addr;
    unregister_kprobe(&kp);
    return addr;
}
```

Usage:

```
target_func_addr = kprobe_lookup("do_sys_openat2");
/* Same result as the old:
 * kallsyms_lookup_name("do_sys_openat2")
 * but works on kernel 5.7+ */
```

Workaround 2: The `sprint_symbol()`

If kprobes are unavailable (`CONFIG_KPROBES=n` or non-GPL module), there's a harder path that works entirely from kernel space.

`sprint_symbol(buf, addr)` is **still exported** — it does address → name (the reverse direction). Brute-force the forward direction:

```
/* Get kernel base from any exported symbol.
 * KASLR randomizes the base, not offsets
 * between functions. */
kaddr = (unsigned long)&sprint_symbol;
kaddr &= 0xffffffffffff0000;

for (i = 0; i < 0x100000; i++) {
    sprint_symbol(buf, kaddr);
    if (strncmp(buf, target, len) == 0)
        return kaddr;
    kaddr += 0x10; /* function-aligned */
}
```

No GPL required — `sprint_symbol` is a regular export.

sprint_symbol

1. `&sprint_symbol` gives you an address in kernel text (e.g., `0xffff800080012340`)
2. `sprint_symbol` is close to the kernel load address (`_text` on arm64)
3. Mask off the low bytes → approximate `_text`
4. Scan forward at function-aligned intervals, calling `sprint_symbol()` at each address
5. When the returned name matches your target, you have the address

Tradeoffs

	Kprobe trick	sprint_symbol() scan
Speed	Instant	Slow (linear scan)
GPL required	Yes	No
Kprobes required	Yes	No
Elegance	Clean	Brute force
Reliability	High	Depends on alignment

Workaround 3: /proc/kallsyms (Userspace)

The most direct approach but requires a **userspace helper**.

/proc/kallsyms exposes the same symbol table that `kallsyms_lookup_name()` queries internally. Parse it from userspace, pass addresses into the module via `module_param`:

```
#!/bin/bash
# resolve.sh - find symbol addresses
KSYM=/proc/kallsyms

lookup() {
    grep " [Tt] `$1" `${KSYM} | awk '{print $1}'
}

KALLSYMS=0x$(lookup kallsyms_lookup_name)
FTREG=0x$(lookup register_ftrace_function)
FTUNREG=0x$(lookup unregister_ftrace_function)
FTFILT=0x$(lookup ftrace_set_filter_ip)

insmod hooklab_nogpl.ko \
    kallsyms_addr=$KALLSYMS \
    ft_register_addr=$FTREG \
    ft_unregister_addr=$FTUNREG \
    ft_filter_addr=$FTFILT
```

inside the module

Inside the module:

```
static unsigned long kallsyms_addr = 0;
module_param(kallsyms_addr, ulong, 0);

typedef unsigned long
    (*lookup_t)(const char *name);

static int __init nogpl_init(void)
{
    lookup_t my_lookup =
        (lookup_t)kallsyms_addr;

    /* Now resolve anything else */
    unsigned long addr =
        my_lookup("do_sys_openat2");
    ...
}
```

Tradeoffs cont

	Kprobe	sprint_symbol	/proc/kallsyms
GPL	Yes	No	No
Kprobes	Yes	No	No
Userspace helper	No	No	Yes
Speed	Pay once	Slow	Fast
Survives KASLR	Yes	Yes	Yes (re-parse each boot)

This is the HW4 challenge mode approach.

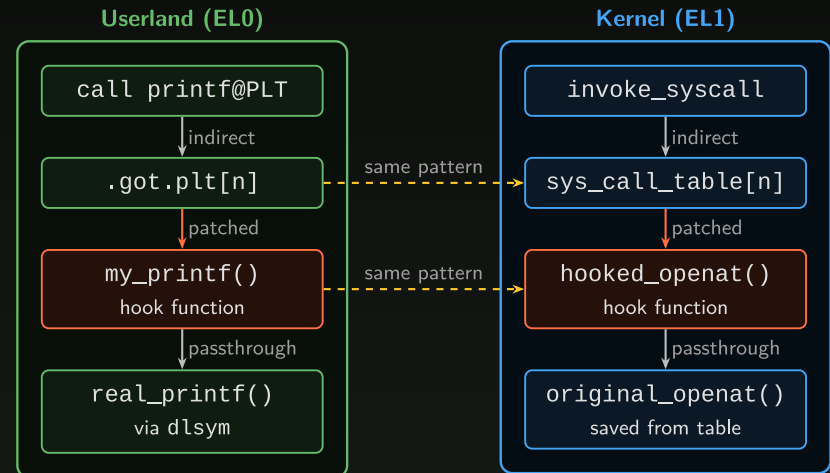
What if there is no kallsyms?

- Don't worry, there is.
- While some exotic systems don't have kallsym, that usually means its a custom kernel
- If you come across one of these systems, your best bet is to find an anchor point along a chain with a deterministic call path, and then pattern match known bytes.
- Or scan kernel for xrefs to strings known to show up in specific kernel code (also painful)

Syscall Table Hooking: Kernel GOT Patching

Today's Agenda:

- The GOT ↔ `sys_call_table` analogy
- Symbol resolution: the kprobe bootstrap trick
- PTE manipulation: making `.rodata` writable
- AArch64 page table walk: PGD → PMD → PTE
- The hook function: direct `pt_regs` access
- Detection: the easiest hook to catch
- Modern mitigations: why this technique is dying



Three Hook Mechanisms

Each level uses the same pattern — **intercept + redirect** — at different privilege levels:

Level	HW	Technique	Mechanism	Privilege
Userspace	HW3	LD_PRELOAD	GOT/PLT patching	EL0
Kernel (probe)	HW4 P2	Kprobes	BRK → exception handler	EL1
Kernel (trace)	HW4 P3	Ftrace	NOP → BL patching	EL1
Kernel (table)	HW4 P4	Syscall table	Pointer replacement	EL1

Overview

	Kprobes	Ftrace	Syscall Table
Install	<code>register_kprobe()</code>	resolve + filter + register	PTE manipulation + pointer write
Kernel support	Supported API	Supported API	Unsupported hack
Arg access	Double <code>pt_regs</code>	Direct (inner function)	Direct <code>pt_regs</code> (single)
Block method	<code>user_regs->regs[1] = 0</code>	<code>fregs - >regs[1] = 0</code>	Return -EACCES directly
Scope	Any instruction	Function entry	Syscall entry only

The GOT ↔ Syscall Table Analogy

From `syscall_hook.c` — the central analogy that ties HW3 to HW4:

	Userland GOT hook	Kernel syscall table hook
Dispatch	<code>.got.plt[]</code>	<code>sys_call_table[]</code>
Make RW	<code>mprotect(PROT_RW)</code>	Clear <code>PTE_RDONLY</code>
Install	<code>GOT[n] = &hook</code>	<code>table[__NR_openat] = &hook</code>
Original	<code>dlsym(RTLD_NEXT)</code>	Saved from <code>table[n]</code>
Restore	<code>GOT[n] = original</code>	<code>table[n] = original</code>

Both are **dispatch table patching**: replace a function pointer in an array the runtime uses for indirect dispatch. The only difference is privilege level and protection mechanism.

HW3: you patched the GOT from within the process (`LD_PRELOAD`). HW4 Part 4: you patch the syscall table from within the kernel (LKM).

extern sys_call_table?

1. **sys_call_table is not exported** — EXPORT_SYMBOL is not used, so modules can't link to it
2. **kallsyms_lookup_name()** is not exported — removed from module access since ~5.7 (commit 0bd476e6c671)
3. **sys_call_table is data, not code** — kprobes instrument *instructions*, so you can't place a breakpoint on a data symbol

So we need to find the address ourselves!

The Kprobe Bootstrap Trick

Step 1: Resolve `kallsyms_lookup_name` via kprobe

- `kallsyms_lookup_name` is a *function* → kprobe can find it
- Register kprobe, read `kp.addr`, unregister immediately

Step 2: Use `kallsyms_lookup_name` to find everything else

- `sys_call_table`: dispatch table (data)
- `init_mm`: kernel page tables (not exported)

This pattern works for **any** non-exported symbol:

- Functions: kprobe directly, or via kallsyms
- Data: kallsyms only (can't kprobe data)

snippet from solution:

```
static unsigned long kprobe_lookup(  
    const char *name)  
{  
    struct kprobe kp = {  
        .symbol_name = name  
    };  
    unsigned long addr;  
  
    if (register_kprobe(&kp) < 0)  
        return 0;  
    addr = (unsigned long)kp.addr;  
    unregister_kprobe(&kp);  
    return addr;  
}
```

Lines 312-317 (bootstrap):

```
lookup_name = (kallsyms_lookup_name_t)  
    kprobe_lookup("kallsyms_lookup_name");
```

Resolving Non-Exported Symbols

Once you have `kallsyms_lookup_name`, any kernel symbol is reachable:

```
/* Step 2: resolve sys_call_table (line 322) */
sys_call_table_ptr = (syscall_fn_t *)lookup_name("sys_call_table");

/* Step 2b: resolve init_mm (line 332) */
p_init_mm = (struct mm_struct *)lookup_name("init_mm");
```

Resolving Non-Exported Symbols

Symbol	Why not exported	Why we need it
<code>sys_call_table</code>	Prevent rootkits	The dispatch table to patch
<code>init_mm</code>	Internal to mm subsystem	PGD root for kernel page table walk
<code>kallsyms_lookup_name</code>	Prevent rootkits (since 5.7)	Resolve everything else

Pattern: **kprobe** → **kallsyms** → **anything**..

Syscall Hook Blueprint

- Find the entry in the table for the target syscall
- save the original address so our hook can call the real function
- Overwrite the value of the table with the virtual address of our hook.
- ...one problem

Table is Read-Only

CONFIG_STRICT_KERNEL_RWX (enabled by default on most kernels):

- Marks kernel .rodata section as **read-only** at the PTE level
- `sys_call_table[]` is declared `const` and placed in .rodata
- `set_memory_rw()` **cannot** flip .rodata: the kernel explicitly refuses

WE NEED TO GO DEEPER

imgflip.com

We are the kernel...

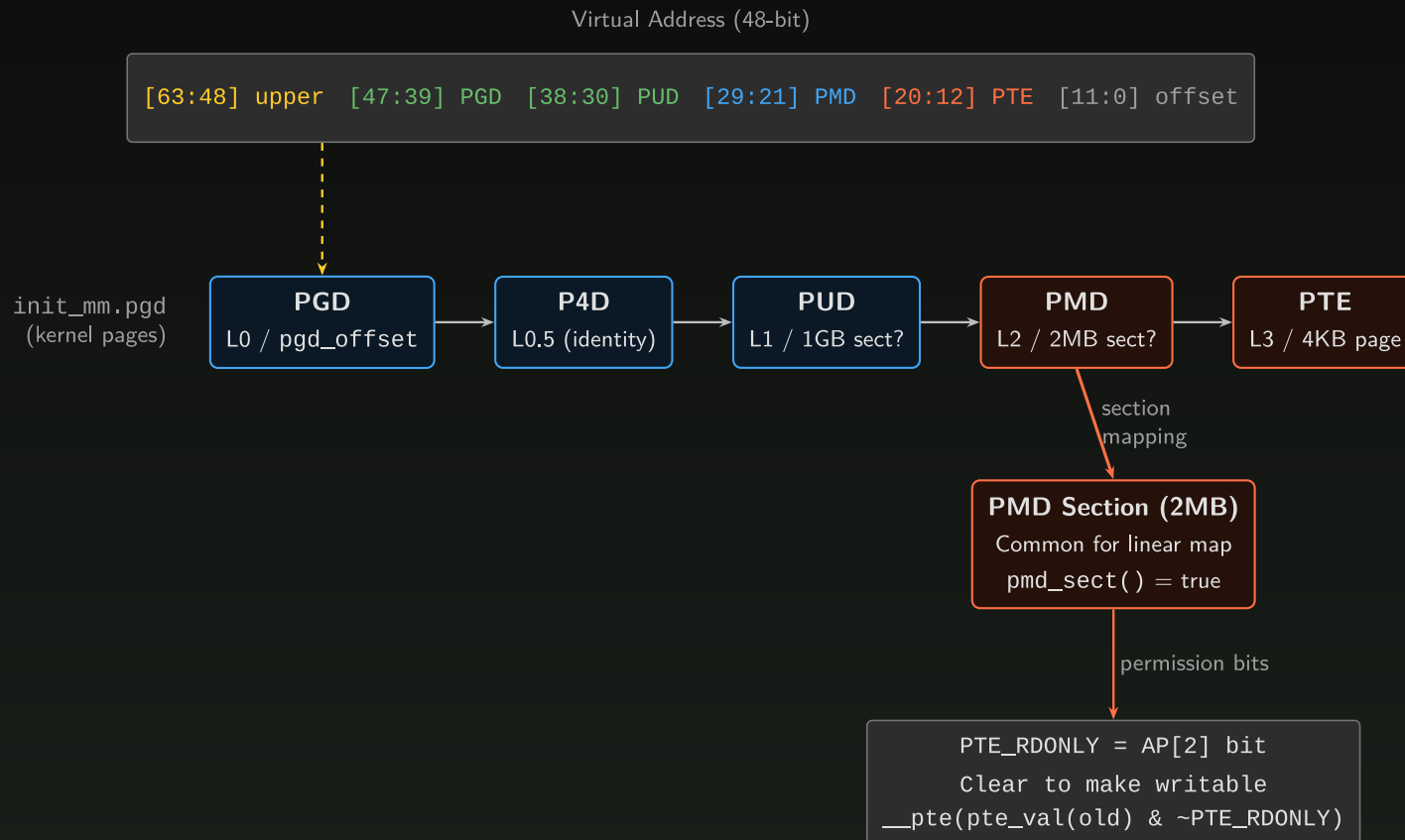
We can directly modify the page table entry (PTE) that maps `sys_call_table`.

Userland analogy:

```
// Userland: change GOT page permissions
mprotect(got_page, PAGE_SIZE, PROT_READ | PROT_WRITE);

// Kernel: we manipulate the PTE that mprotect would change
// but for kernel memory, bypassing all kernel protections
```

AArch64 Page Table Walk



Don't worry, we will cover this in depth next class.

Walking the Page Table

- How do we find the Page table root?
- What is the structure of the page table?
- "Wait didn't you say something about caches last time?"
- "WTF is a page table?"
- "MMU????"

Page Tables

- "The thing that actually gives us Virtual memory"
- Organizes memory into chunks called pages
- Physical pages: logical organization of physical memory
 - identify by its Physical Page number (PPN)
- Virtual Page:
 - Identify by Virtual Page Number (VPN)
- "Its basically a radix tree"
- Built in software
- Walked in hardware

OK but how do we actually walk it

PGD → P4D → PUD → PMD → PTE

Title

- **PGD = Page Global Directory** (top-level table)
- **P4D = Page 4th Directory** (next level; may be folded on some configs)
 - The kernel still exposes the 5-level API (PGD → P4D → PUD → PMD → PTE) for portability.
 - But P4D is effectively an alias/pass-through to the next real level.
- **PUD = Page Upper Directory**
- **PMD = Page Middle Directory**
- **PTE = Page Table Entry** (leaf entry mapping a page)

Translate VA to PA

Each level indexes into the next using bits from the virtual address. At each level, check for:

- `*_none()` — entry not present → fail
- `pud_sect()` — 1GB section (rare)
- `pmd_sect()` — 2MB section (common!)

On AArch64, the kernel linear map typically uses **2MB PMD section mappings**. This means the walk stops at PMD level — there is no separate PTE.

PMD section descriptors use the **same permission bit layout** as PTE descriptors, so we cast `pmd_t *` → `pte_t *` and manipulate it uniformly.

Clearing PTE_RDONLY

AArch64 AP[2] bit = PTE_RDONLY

To make the table writable:

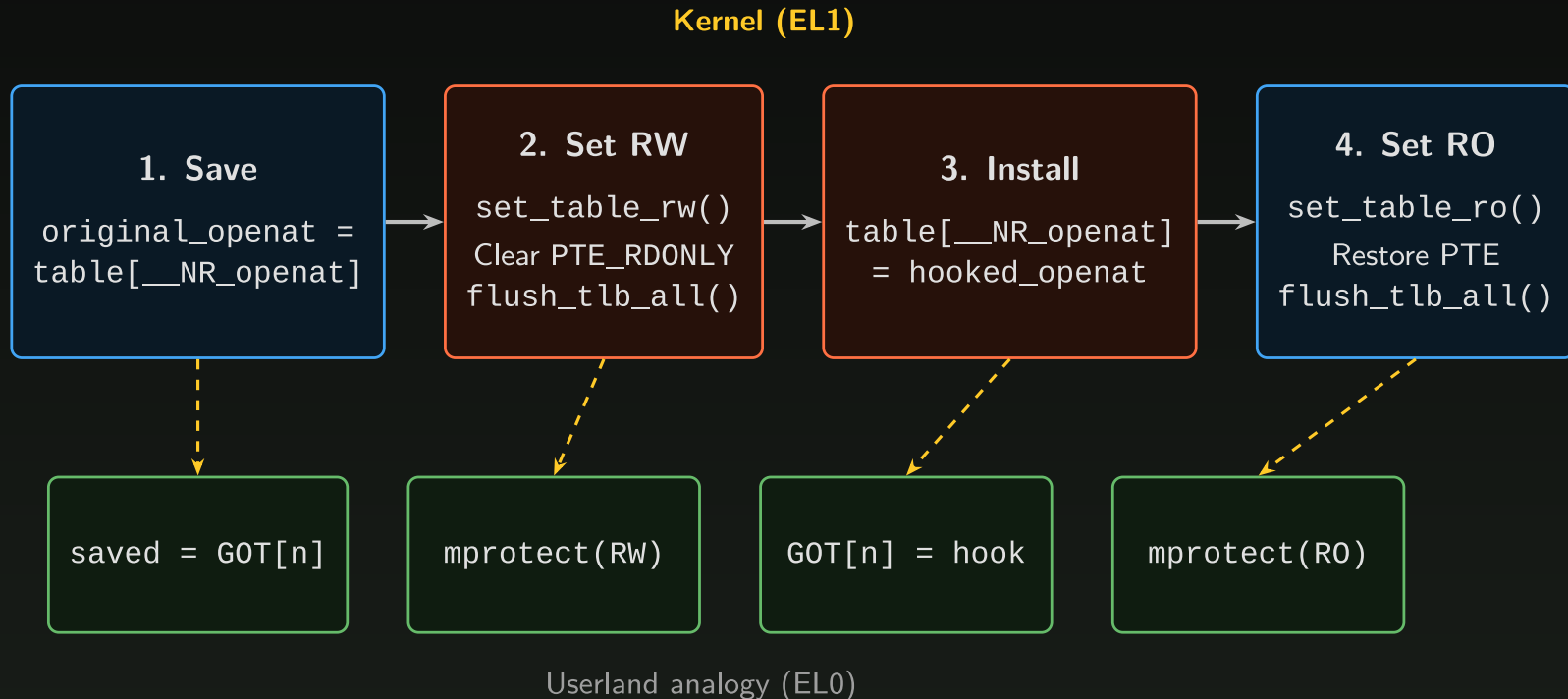
1. Walk page tables → get pointer to PTE/PMD entry
2. Save the original PTE value
3. Clear the read-only bit: `pte_val(old) & ~PTE_RDONLY`
4. `WRITE_ONCE` the new PTE
5. `flush_tlb_all()` so all CPUs see the change

To restore protection:

1. `WRITE_ONCE` the saved PTE back
2. `flush_tlb_all()` again

WRITE_ONCE prevents compiler reordering. **flush_tlb_all()** invalidates TLB on all CPUs — otherwise stale cached translations would still allow writes.

The Full RW → Write → RO Sequence

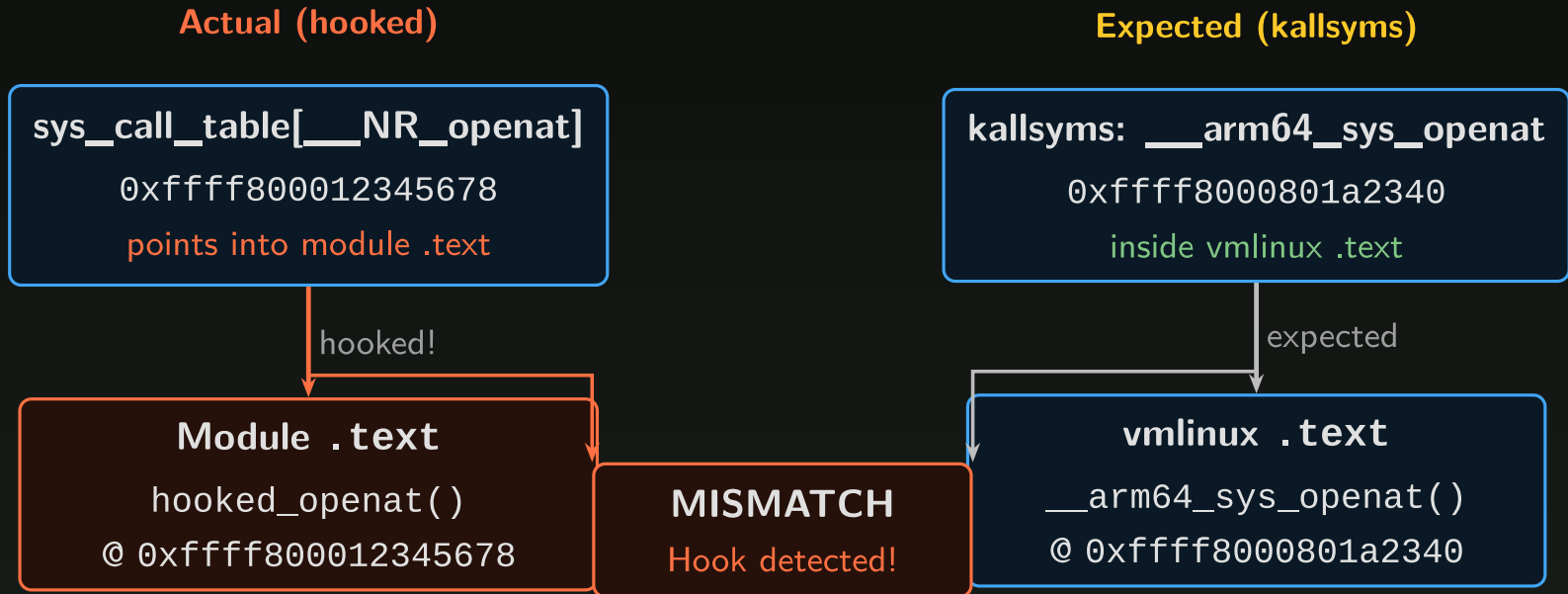


The window of vulnerability is tiny: the table is writable only between `set_table_rw()` and `set_table_ro()`. But during that window, **any CPU** could write to the 2MB section containing the table.

Three Ways to Block: Compared

- The kprobe/ftrace technique is a **side effect** — you corrupt an argument and let the kernel trip over it.
- Table hooking is a **direct replacement** — you *are* the syscall handler and return whatever you want.

Detecting Syscall Table Hooks



Detection Script

- From userspace compare `sys_call_table` entries against `/proc/kallsyms`
- From a detection kernel module:

```
for (int i = 0; i < NR_syscalls; i++) {  
    unsigned long addr = (unsigned long)sys_call_table_ptr[i];  
    if (!core_kernel_text(addr))  
        pr_alert("sys_call_table[%d] points outside vmlinux .text!\n", i);  
}
```

`core_kernel_text()` returns true only for addresses inside the main kernel image. Any entry pointing into module space is suspicious.

Detection Is Trivial

The syscall table is a **global array at a known address**. Every entry should point to a known symbol in the `vmlinux .text` section.

1. **Fixed address:** `sys_call_table` address is in `/proc/kallsyms`
2. **Known values:** every entry should match a `__arm64_sys_*` symbol
3. **Address range check:** `module .text` lives at a different virtual address range than `vmlinux .text` — `core_kernel_text()` catches this instantly

Unlike kprobes (which patch single instructions) or ftrace (which patches call sites), syscall table hooking modifies a **data structure** that is trivially enumerable.

There is no kernel-maintained list of "legitimate table modifications" because there are no legitimate table modifications.

Detection is trivial when you look

All three are easy to detect. The difference:

- **Kprobes/ftrace** have legitimate users as a detection tool must distinguish malicious hooks from system tracers
- **Syscall table patching** has **no** legitimate users as any modification is inherently suspicious

This is why syscall table hooking is the worst choice for stealth.

CONFIG_STRICT_KERNEL_RWX

The first line of defense:

- Makes `.rodata` (including `sys_call_table`) **read-only** at the PTE level
- Enabled by default on all major distros
- Our module defeats it with direct PTE manipulation — but we're fighting the kernel

What we do to bypass it:

```
new = __pte(pte_val(saved_pte) & ~PTE_RDONLY);  
WRITE_ONCE(*saved_ptep, new);  
flush_tlb_all();
```

In practice

What newer kernels add:

- **Pinned PTEs:** prevent modification of page table entries for kernel text/rodata
- **CMA guard pages:** isolate sensitive kernel structures
- **RO page table entries:** mark the PTEs themselves as read-only (recursive protection)
- Hardmode: Hypervisor protecting PTEs.
- Hardmode: Hardware feature protecting PTEs.

Each generation makes PTE manipulation harder. Our technique works on current lab kernels but becomes increasingly fragile on newer ones.

